

PCCST503 – Machine Learning

Assignment 1

Design of a Safe Semantic Planner in a Finite Cartesian State Space

Design Report

Implementation: C++17

1. Introduction

The assignment requires a generic planner that computes a safe path in a finite Cartesian state space. The planner receives an initial state, a goal state, bad states, and directed transitions. Each transition contains cost, reliability, safety information, and an availability flag. The implementation provides graph search, safety checking, path reconstruction, dynamic updates, and experimental measurements.

2. Problem Definition

The state space is a finite set of states embedded in R^d . Each state is represented by a numeric vector. A planning problem contains an initial state, a goal state, a set of bad states, states, and directed transitions. A valid solution reaches the goal without visiting bad states, minimizes transition cost, and seeks a large minimum Euclidean distance from bad states.

3. State Representation

Each State contains a 64-bit identifier and a vector embedding. The identifier is the graph key and the embedding stores Cartesian coordinates used for Euclidean-distance calculations.

4. Transition Representation

Each Transition stores an identifier, source state, destination state, cost, safety value, reliability, and availability flag. Transitions are directed. Unavailable transitions are ignored and destinations marked as bad are not traversed.

5. Data Structures

States and transitions are indexed with hash maps for direct lookup. An outgoing-transition index maps each source state to transition identifiers. A priority queue maintains the search frontier. Parent-state and parent-transition maps reconstruct the final path.

6. Algorithm Design

The implementation follows the LPA* family suggested by the assignment. Cost information is propagated from the goal toward the initial state through available directed transitions. A priority queue orders search states using accumulated effective cost and a Euclidean heuristic. Parent links are then followed from the initial state to reconstruct the solution.

7. Heuristic Function

The implementation uses Euclidean distance between Cartesian embeddings as the search heuristic. For vectors x and y , the distance is $\sqrt{\sum (x_i - y_i)^2}$. This uses the finite Cartesian representation required by the assignment.

8. Safety Computation

For each state, the planner computes its distance to every bad state and uses the minimum distance. The path safety score is the minimum such distance over the selected path. A transition leading to a bad state is excluded. Safety also contributes a penalty to effective search cost.

9. Cost and Reliability

Raw path cost is the sum of transition costs. Cumulative reliability is the product of the reliabilities of selected transitions. The search objective uses raw cost together with safety and reliability penalties, while the result reports raw cost, safety distance, and cumulative reliability separately.

10. Dynamic Replanning

The Planner provides `plan()` and `replan()` operations. Replanning accepts the updated planning problem and searches the updated graph. This supports goal changes, bad-state changes, transition availability changes, removed transitions, and newly added transitions. Replanning time is measured separately.

11. Complexity Analysis

Let V be the number of states and E the number of directed transitions. Building the indexes is $O(V+E)$. The priority-queue search is $O((V+E) \log V)$ in the usual binary-heap model, subject to state-update behavior. If B bad states are present, evaluating distances can require $O(VB)$ work in the worst case. Graph and search storage is $O(V+E)$, with additional parent and queue storage.

12. Test Cases

Test	Scenario	Observed result
1	Basic Reachability	$1 \rightarrow 2 \rightarrow 3 \rightarrow 4$
2	Bad State Avoidance	$1 \rightarrow 5 \rightarrow 6 \rightarrow 4$
3	Safety Margin	$1 \rightarrow 5 \rightarrow 6 \rightarrow 4$
4	Dynamic Transition	Alternative path after removal
5	Goal Update	Path recalculated to new goal
6	Transition Addition	Shortcut $1 \rightarrow 4$ discovered

13. Experimental Results

Case	Success	Cost	Minimum safety	Explored
1	Yes	6.0	No bad states	6
2	Yes	9.0	1.4142	5
3	Yes	9.0	1.2806	6
4B	Yes	4.0	No bad states	3
5B	Yes	6.0	No bad states	3
6B	Yes	1.0	No bad states	6

The six required scenarios executed successfully in the submitted implementation. Bad-state avoidance selected the safe alternative; transition removal caused replanning; goal modification produced a revised path; and transition addition exposed the shortcut. Runtime values are machine-dependent and are also written to `results.csv`.

14. Software Structure

`State.h` defines Cartesian states; `Transition.h` defines directed transitions; `PlanningProblem.h` packages inputs; `PlanningResult.h` stores outputs and metrics; `Planner.h/.cpp` implement planning and replanning; `main.cpp` executes the six scenarios and writes experimental results. `README.md` and `docs/USER_MANUAL.md` provide usage information.

15. Conclusion

The implementation provides a C++17 safe semantic planner for a finite Cartesian state space. It represents states and directed transitions, avoids bad and unavailable states, evaluates safety and reliability, supports dynamic replanning, and records experimental measurements for the assignment deliverables.